

Установка и настройка

Установка библиотеки

pip install psycopg2-binary

Для production-окружения рекомендуется использовать полную версию:

pip install psycopg2

Системные требования

Перед установкой убедитесь, что в системе установлены необходимые зависимости:

Ubuntu/Debian:

sudo apt-get install libpq-dev python3-dev

CentOS/RHEL:

sudo yum install postgresql-devel python3-devel

macOS:

brew install postgresql

Подключение к базе данных

Базовое подключение

import psycopg2

Подключение к PostgreSQL

```
conn = psycopg2.connect(  
    dbname="mydb",  
    user="postgres",  
    password="password",  
    host="localhost",
```

```
port="5432"
)

# Создание курсора
cur = conn.cursor()

Подключение через строку подключения
conn = psycopg2.connect("postgresql://user:password@localhost:5432/dbname")

Подключение с дополнительными параметрами
conn = psycopg2.connect(
    dbname="mydb",
    user="postgres",
    password="password",
    host="localhost",
    port="5432",
    sslmode="require",
    connect_timeout=10,
    application_name="my_app"
)
```

Использование контекстных менеджеров

```
import psycopg2
from contextlib import contextmanager
```

```
@contextmanager
def get_db_connection():
    conn = psycopg2.connect(
        dbname="mydb",
        user="postgres",
        password="password",
```

```
        host="localhost"
    )
    try:
        yield conn
    finally:
        conn.close()
```

Использование

```
with get_db_connection() as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM users")
        results = cur.fetchall()
```

Создание базы данных и таблиц

Создание таблицы

```
cur.execute("""
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    age INTEGER CHECK (age > 0),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE
)
""")
conn.commit()
```

Создание индексов

```
cur.execute("CREATE INDEX IF NOT EXISTS idx_users_email ON users(email)")
cur.execute("CREATE INDEX IF NOT EXISTS idx_users_created_at ON users(created_at)")
```

```
conn.commit()
```

Создание составных таблиц

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS orders (
```

```
    id SERIAL PRIMARY KEY,
```

```
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
```

```
    product_name VARCHAR(200) NOT NULL,
```

```
    quantity INTEGER NOT NULL DEFAULT 1,
```

```
    price DECIMAL(10,2) NOT NULL,
```

```
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    metadata JSONB
```

```
)
```

```
""")
```

```
conn.commit()
```

CRUD-операции с Psycopg2

Добавление записей

Одиночная вставка

```
cur.execute(
```

```
    "INSERT INTO users (name, email, age) VALUES (%s, %s, %s)",
```

```
    ("Иван Иванов", "ivan@example.com", 30)
```

```
)
```

```
conn.commit()
```

Получение ID вставленной записи

```
cur.execute(
```

```
    "INSERT INTO users (name, email, age) VALUES (%s, %s, %s) RETURNING id",
```

```
    ("Петр Петров", "petr@example.com", 25)
```

```
)
```

```
user_id = cur.fetchone()[0]
```

```
conn.commit()
```

Массовая вставка

```
users_data = [  
    ("Анна Смирнова", "anna@example.com", 28),  
    ("Максим Козлов", "maxim@example.com", 35),  
    ("Елена Васильева", "elena@example.com", 24)  
]
```

```
cur.executemany(  
    "INSERT INTO users (name, email, age) VALUES (%s, %s, %s)",  
    users_data  
)  
conn.commit()
```

Эффективная массовая вставка

```
from psycopg2.extras import execute_values
```

```
execute_values(  
    cur,  
    "INSERT INTO users (name, email, age) VALUES %s",  
    users_data,  
    template=None,  
    page_size=100  
)  
conn.commit()
```

Выборка данных

Базовая выборка

```
cur.execute("SELECT id, name, email, age FROM users")  
rows = cur.fetchall()
```

for row in rows:

print(f"ID: {row[0]}, Имя: {row[1]}, Email: {row[2]}, Возраст: {row[3]}")

Выборка с условиями

cur.execute("SELECT * FROM users WHERE age > %s AND is_active = %s", (25, True))

active_users = cur.fetchall()

Выборка с сортировкой и ограничением

cur.execute("""

SELECT name, email, age

FROM users

WHERE age BETWEEN %s AND %s

ORDER BY age DESC

LIMIT %s

""", (20, 50, 10))

Использование fetchone() и fetchmany()

Получение одной записи

cur.execute("SELECT * FROM users WHERE id = %s", (1,))

user = cur.fetchone()

Получение нескольких записей

cur.execute("SELECT * FROM users")

first_five = cur.fetchmany(5)

Итерация по результатам

cur.execute("SELECT * FROM users")

for row in cur:

print(row)

Обновление данных

Обновление одной записи

```
cur.execute(  
    "UPDATE users SET email = %s WHERE id = %s",  
    ("new_email@example.com", 1)  
)
```

Обновление нескольких полей

```
cur.execute("""  
UPDATE users  
SET email = %s, age = %s, is_active = %s  
WHERE id = %s  
""", ("updated@example.com", 31, True, 1))
```

Массовое обновление

```
cur.execute(  
    "UPDATE users SET is_active = %s WHERE age < %s",  
    (False, 18)  
)
```

conn.commit()

Удаление данных

Удаление по ID

```
cur.execute("DELETE FROM users WHERE id = %s", (1,))
```

Условное удаление

```
cur.execute("DELETE FROM users WHERE is_active = %s", (False,))
```

Удаление с возвратом данных

```
cur.execute("DELETE FROM users WHERE age < %s RETURNING id, name", (18,))
```



```

cur.execute("UPDATE users SET age = %s WHERE email = %s",
            (30, "context@example.com"))

# Автоматический commit при выходе из блока

except Exception as e:

    # Автоматический rollback при ошибке

    print(f"Ошибка транзакции: {e}")

Savepoints (точки сохранения)

try:

    with conn:

        with conn.cursor() as cur:

            cur.execute("INSERT INTO users (name, email) VALUES (%s, %s)",
                        ("Пользователь1", "user1@example.com"))

            # Создание savepoint

            cur.execute("SAVEPOINT sp1")

        try:

            cur.execute("INSERT INTO users (name, email) VALUES (%s, %s)",
                        ("Пользователь2", "user2@example.com"))

            cur.execute("INSERT INTO users (name, email) VALUES (%s, %s)",
                        ("Дубликат", "user1@example.com")) # Ошибка дубликата

        except Exception:

            # Откат к savepoint

            cur.execute("ROLLBACK TO SAVEPOINT sp1")

            print("Откат к savepoint")

        cur.execute("INSERT INTO users (name, email) VALUES (%s, %s)",
                    ("Пользователь3", "user3@example.com"))

```

except Exception as e:

```
    print(f"Ошибка: {e}")
```

Чтение результатов запросов

Методы получения данных

Выполнение запроса

```
cur.execute("SELECT id, name, email FROM users LIMIT 10")
```

fetchone() - получить одну строку

```
first_row = cur.fetchone()
```

```
print(first_row) # (1, 'Иван', 'ivan@example.com')
```

fetchmany(n) - получить n строк

```
cur.execute("SELECT id, name, email FROM users")
```

```
five_rows = cur.fetchmany(5)
```

```
for row in five_rows:
```

```
    print(row)
```

fetchall() - получить все строки

```
cur.execute("SELECT id, name, email FROM users")
```

```
all_rows = cur.fetchall()
```

```
print(f"Всего строк: {len(all_rows)}")
```

Итерация по курсору

```
cur.execute("SELECT id, name, email FROM users")
```

```
for row in cur:
```

```
    id, name, email = row
```

```
    print(f"ID: {id}, Имя: {name}, Email: {email}")
```

Работа с именованными курсорами

```
from psycopg2.extras import DictCursor
```

Создание курсора с возможностью доступа по имени колонки

```
cur = conn.cursor(cursor_factory=DictCursor)
```

```
cur.execute("SELECT id, name, email FROM users LIMIT 1")
```

```
row = cur.fetchone()
```

```
print(row['name']) # Доступ по имени колонки
```

```
print(row['email']) # Доступ по имени колонки
```

```
print(row[0])      # Доступ по индексу также работает
```

Серверные курсоры для больших данных

Создание серверного курсора для обработки больших наборов данных

```
cur = conn.cursor(name='large_data_cursor')
```

```
cur.execute("SELECT * FROM large_table")
```

Обработка данных порциями

```
while True:
```

```
    rows = cur.fetchmany(1000)
```

```
    if not rows:
```

```
        break
```

```
    for row in rows:
```

```
        # Обработка каждой строки
```

```
        process_row(row)
```

```
cur.close()
```

Безопасность и защита от SQL-инъекций

Правильное использование параметров

ПРАВИЛЬНО - использование параметризованных запросов

```
email = "user@example.com"
```

```
cur.execute("SELECT * FROM users WHERE email = %s", (email,))
```

ПРАВИЛЬНО - множественные параметры

```
cur.execute("SELECT * FROM users WHERE age > %s AND email = %s", (25, email))
```

ПРАВИЛЬНО - именованные параметры

```
cur.execute("SELECT * FROM users WHERE age > %(min_age)s AND email = %(email)s",  
            {'min_age': 25, 'email': email})
```

Что НЕ следует делать

НЕПРАВИЛЬНО - уязвимость к SQL-инъекциям

```
email = "user@example.com"
```

```
cur.execute(f"SELECT * FROM users WHERE email = '{email}'")
```

НЕПРАВИЛЬНО - использование .format()

```
cur.execute("SELECT * FROM users WHERE email = '{}'.format(email))
```

НЕПРАВИЛЬНО - использование % для строк

```
cur.execute("SELECT * FROM users WHERE email = '%s'" % email)
```

Экранирование идентификаторов

```
from psycopg2.sql import SQL, Identifier, Literal
```

Безопасное создание динамических запросов

```
table_name = "users"
```

```
column_name = "email"
```

```
query = SQL("SELECT * FROM {} WHERE {} = %s").format(  
    Identifier(table_name),
```

```
    Identifier(column_name)
)
```

```
cur.execute(query, ("user@example.com",))
```

Композиция сложных запросов

```
from psycopg2.sql import SQL, Identifier, Literal, Composed
```

Построение сложного запроса

```
def build_select_query(table, columns, where_conditions):
```

```
    query = SQL("SELECT {} FROM {}").format(
        SQL(', ').join(map(Identifier, columns)),
        Identifier(table)
    )
```

```
    if where_conditions:
```

```
        where_clause = SQL(" AND ").join(
            SQL("{} = %s").format(Identifier(col))
            for col in where_conditions.keys()
        )
```

```
        query = SQL("{} WHERE {}").format(query, where_clause)
```

```
    return query
```

Использование

```
query = build_select_query(
    'users',
    ['id', 'name', 'email'],
    {'age': 25, 'is_active': True}
```

)

```
cur.execute(query, (25, True))
```

Работа с типами данных PostgreSQL

Автоматическое преобразование типов

Psycopg2 автоматически преобразует типы данных между PostgreSQL и Python:

Тип PostgreSQL	Тип Python	Пример
INTEGER, SERIAL	int	42
VARCHAR, TEXT	str	"строка"
BOOLEAN	bool	True
DECIMAL, NUMERIC	decimal.Decimal	Decimal('123.45')
TIMESTAMP	datetime.datetime	datetime.now()
DATE	datetime.date	date.today()
TIME	datetime.time	time(14, 30)
JSON, JSONB	dict, list	{"key": "value"}
UUID	uuid.UUID	uuid.uuid4()
BYTEA	bytes	b"binary data"
ARRAY	list	[1, 2, 3]

Работа с JSON данными

```
import json
```

```
from psycopg2.extras import Json
```

Создание таблицы с JSON

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS logs (
```

```
    id SERIAL PRIMARY KEY,
```

```
    message TEXT,
```

```
    metadata JSONB,
```

```
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
    """)
```

Вставка JSON данных

```
metadata = {
    "ip": "192.168.1.1",
    "user_agent": "Mozilla/5.0...",
    "session_id": "abc123"
}
```

```
cur.execute(
    "INSERT INTO logs (message, metadata) VALUES (%s, %s)",
    ("Пользователь вошел в систему", Json(metadata))
)
```

Запрос JSON данных

```
cur.execute("SELECT metadata FROM logs WHERE id = %s", (1,))
row = cur.fetchone()
print(row[0]['ip']) # Автоматическое преобразование в dict
```

Работа с массивами PostgreSQL

Создание таблицы с массивом

```
cur.execute("""
CREATE TABLE IF NOT EXISTS products (
    id SERIAL PRIMARY KEY,
    name TEXT,
    tags TEXT[],
    prices DECIMAL[]
)
```

```
)  
""")
```

Вставка массивов

```
cur.execute(  
    "INSERT INTO products (name, tags, prices) VALUES (%s, %s, %s)",  
    ("Ноутбук", ["электроника", "компьютер"], [50000.00, 45000.00])  
)
```

Поиск по массиву

```
cur.execute("SELECT * FROM products WHERE %s = ANY(tags)", ("электроника",))
```

Работа с UUID

```
import uuid
```

Создание таблицы с UUID

```
cur.execute("""  
CREATE TABLE IF NOT EXISTS sessions (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id INTEGER,  
    token UUID NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
)  
""")
```

Вставка UUID

```
session_token = uuid.uuid4()
```

```
cur.execute(  
    "INSERT INTO sessions (user_id, token) VALUES (%s, %s)",
```



```
(1, session_token)  
)
```

Поиск по UUID

```
cur.execute("SELECT * FROM sessions WHERE token = %s", (session_token,))
```

Работа с временными типами

```
from datetime import datetime, date, time
```

Создание таблицы с временными типами

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS events (
```

```
    id SERIAL PRIMARY KEY,
```

```
    title TEXT,
```

```
    event_date DATE,
```

```
    event_time TIME,
```

```
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```
)
```

```
""")
```

Вставка временных данных

```
cur.execute(
```

```
    "INSERT INTO events (title, event_date, event_time) VALUES (%s, %s, %s)",
```

```
    ("Встреча", date(2024, 12, 25), time(14, 30))
```

```
)
```

Работа с временными зонами

```
cur.execute("SELECT created_at AT TIME ZONE 'UTC' FROM events")
```

Продвинутые возможности Psycopg2

Работа с хранимыми процедурами

Создание хранимой процедуры

```
cur.execute("""
```

```
CREATE OR REPLACE FUNCTION get_user_stats(user_id INTEGER)
```

```
RETURNS TABLE(total_orders INTEGER, total_amount DECIMAL) AS $$
```

```
BEGIN
```

```
    RETURN QUERY
```

```
    SELECT COUNT(*)::INTEGER, COALESCE(SUM(price * quantity), 0)::DECIMAL
```

```
    FROM orders
```

```
    WHERE orders.user_id = get_user_stats.user_id;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
""")
```

Вызов хранимой процедуры

```
cur.callproc('get_user_stats', (1,))
```

```
result = cur.fetchone()
```

```
print(f"Заказов: {result[0]}, Сумма: {result[1]}")
```

Работа с большими объектами (Large Objects)

Создание большого объекта

```
large_obj = conn.lobject(mode='w')
```

```
large_obj.write(b"Большой объем данных...")
```

```
loid = large_obj.oid
```

```
large_obj.close()
```

Сохранение ссылки на объект

```
cur.execute("INSERT INTO files (name, data_oid) VALUES (%s, %s)",
```

```
    ("document.pdf", loid))
```

Чтение большого объекта

```
cur.execute("SELECT data_oid FROM files WHERE name = %s", ("document.pdf",))
```

```
oid = cur.fetchone()[0]
```

```
large_obj = conn.lobject(oid, mode='r')
```

```
data = large_obj.read()
```

```
large_obj.close()
```

Асинхронные уведомления

Подписка на уведомления

```
cur.execute("LISTEN user_updates")
```

```
conn.commit()
```

Отправка уведомления (из другого соединения)

```
cur.execute("NOTIFY user_updates, 'Новый пользователь создан'")
```

```
conn.commit()
```

Проверка уведомлений

```
conn.poll()
```

```
while conn.notifies:
```

```
    notify = conn.notifies.pop(0)
```

```
    print(f"Получено уведомление: {notify.channel} - {notify.payload}")
```

Копирование данных

```
from io import StringIO
```

COPY TO - экспорт данных

```
cur.execute("COPY users TO STDOUT WITH CSV HEADER")
```

```
with open('users_export.csv', 'w') as f:
```

```
    while True:
```

```
data = cur.fetchone()
```

```
if data is None:
```

```
    break
```

```
f.write(data)
```

COPY FROM - импорт данных

with open('users_import.csv', 'r') as f:

```
    cur.copy_from(f, 'users', columns=('name', 'email', 'age'), sep=',')
```

Полная таблица методов и функций Psycopg2

Основные методы подключения

Метод/Функция	Описание	Пример использования
---------------	----------	----------------------

<code>psycopg2.connect()</code>	Создание соединения с базой данных	<code>conn = psycopg2.connect(dsn)</code>
---------------------------------	------------------------------------	---

<code>conn.cursor()</code>	Создание курсора для выполнения запросов	<code>cur = conn.cursor()</code>
----------------------------	--	----------------------------------

<code>conn.close()</code>	Заккрытие соединения	<code>conn.close()</code>
---------------------------	----------------------	---------------------------

<code>conn.commit()</code>	Подтверждение транзакции	<code>conn.commit()</code>
----------------------------	--------------------------	----------------------------

<code>conn.rollback()</code>	Откат транзакции	<code>conn.rollback()</code>
------------------------------	------------------	------------------------------

Методы курсора

Метод	Описание	Пример использования
-------	----------	----------------------

<code>cur.execute(sql, vars)</code>	Выполнение SQL-запроса	<code>cur.execute("SELECT * FROM users WHERE id = %s", (1,))</code>
-------------------------------------	------------------------	---

<code>cur.executemany(sql, vars_list)</code>	Выполнение запроса для множества параметров	<code>cur.executemany("INSERT INTO users (name) VALUES (%s)", [("John",), ("Jane",)])</code>
--	---	--

<code>cur.fetchone()</code>	Получение одной записи	<code>row = cur.fetchone()</code>
-----------------------------	------------------------	-----------------------------------

<code>cur.fetchmany(size)</code>	Получение нескольких записей	<code>rows = cur.fetchmany(10)</code>
----------------------------------	------------------------------	---------------------------------------

<code>cur.fetchall()</code>	Получение всех записей	<code>all_rows = cur.fetchall()</code>
-----------------------------	------------------------	--

<code>cur.callproc(procname, vars)</code>	Вызов хранимой процедуры	<code>cur.callproc('my_proc', (param1, param2))</code>
---	--------------------------	--

<code>cur.close()</code>	Заккрытие курсора	<code>cur.close()</code>
--------------------------	-------------------	--------------------------

`cur.copy_from(file, table)` Копирование данных из файла `cur.copy_from(f, 'users', columns=('name', 'email'))`

`cur.copy_to(file, table)` Копирование данных в файл `cur.copy_to(f, 'users')`

Атрибуты курсора

Атрибут	Описание	Пример использования
---------	----------	----------------------

<code>cur.description</code>	Описание колонок результата	<code>columns = [desc[0] for desc in cur.description]</code>
------------------------------	-----------------------------	--

<code>cur.rowcount</code>	Количество затронутых строк	<code>print(f"Обновлено строк: {cur.rowcount}")</code>
---------------------------	-----------------------------	--

<code>cur.rownumber</code>	Текущий номер строки	<code>print(f"Текущая строка: {cur.rownumber}")</code>
----------------------------	----------------------	--

<code>cur.query</code>	Последний выполненный запрос	<code>print(cur.query)</code>
------------------------	------------------------------	-------------------------------

<code>cur.statusmessage</code>	Сообщение о статусе операции	<code>print(cur.statusmessage)</code>
--------------------------------	------------------------------	---------------------------------------

Методы управления транзакциями

Метод	Описание	Пример использования
-------	----------	----------------------

<code>conn.autocommit</code>	Включение/выключение автокоммита	<code>conn.autocommit = True</code>
------------------------------	----------------------------------	-------------------------------------

<code>conn.get_transaction_status()</code>	Получение статуса транзакции	<code>status = conn.get_transaction_status()</code>
--	------------------------------	---

<code>conn.set_isolation_level(level)</code>	Установка уровня изоляции	<code>conn.set_isolation_level(psycopg2.extensions.ISOLATION_LEVEL_SERIALIZABLE)</code>
--	---------------------------	---

<code>conn.set_session(readonly=True)</code>	Настройка параметров сессии	<code>conn.set_session(readonly=True, deferrable=True)</code>
--	-----------------------------	---

Дополнительные модули и функции

Модуль/Функция	Описание	Пример использования
----------------	----------	----------------------

<code>psycopg2.extras.DictCursor</code>	Курсор с доступом к колонкам по имени	<code>cur = conn.cursor(cursor_factory=DictCursor)</code>
---	---------------------------------------	---

<code>psycopg2.extras.RealDictCursor</code>	Курсор, возвращающий настоящие словари	<code>cur = conn.cursor(cursor_factory=RealDictCursor)</code>
---	--	---

<code>psycopg2.extras.execute_values()</code>	Эффективная вставка множества значений	<code>execute_values(cur, "INSERT INTO t VALUES %s", data)</code>
---	--	---

<code>psycopg2.extras.execute_batch()</code>	Пакетное выполнение запросов	<code>execute_batch(cur, "INSERT INTO t VALUES (%s, %s)", data)</code>
--	------------------------------	--

psycopg2.extras.Json() Обертка для JSON данных **cur.execute("INSERT INTO t (data) VALUES (%s)", (Json(data),))**

psycopg2.sql.SQL() Композиция SQL-запросов **query = SQL("SELECT * FROM {}".format(Identifier(table)))**

psycopg2.sql.Identifier() Безопасное экранирование идентификаторов
Identifier('table_name')

psycopg2.sql.Literal() Безопасное экранирование литералов **Literal('string_value')**

Обработка ошибок

Исключение **Описание** **Когда возникает**

psycopg2.Error Базовое исключение для всех ошибок Родительский класс для всех ошибок

psycopg2.Warning Предупреждение Уведомления от сервера

psycopg2.InterfaceError Ошибка интерфейса Проблемы с модулем

psycopg2.DatabaseError Ошибка базы данных Проблемы, связанные с базой данных

psycopg2.DataError Ошибка данных Некорректные данные

psycopg2.OperationalError Операционная ошибка Проблемы с соединением, транзакциями

psycopg2.IntegrityError Ошибка целостности Нарушение ограничений

psycopg2.InternalError Внутренняя ошибка Внутренние ошибки PostgreSQL

psycopg2.ProgrammingError Ошибка программирования Синтаксические ошибки SQL

psycopg2.NotSupportedError Неподдерживаемая операция Использование неподдерживаемых возможностей

Интеграция с популярными фреймворками

Интеграция с Pandas

```
import pandas as pd
```

```
import psycopg2
```

```
# Чтение данных из PostgreSQL в DataFrame
```

```
conn = psycopg2.connect(dsn)
```

```
df = pd.read_sql("SELECT * FROM users WHERE age > 25", conn)
```

```
# Запись DataFrame в PostgreSQL
```

```
df.to_sql('users_backup', conn, if_exists='replace', index=False)
```

```
# Использование параметров в запросах
```

```
query = "SELECT * FROM users WHERE age BETWEEN %s AND %s"
```

```
df = pd.read_sql(query, conn, params=(25, 50))
```

```
Интеграция с SQLAlchemy
```

```
from sqlalchemy import create_engine
```

```
import pandas as pd
```

```
# Создание движка SQLAlchemy с psycopg2
```

```
engine = create_engine('postgresql+psycopg2://user:password@localhost/dbname')
```

```
# Использование с Pandas
```

```
df = pd.read_sql("SELECT * FROM users", engine)
```

```
df.to_sql('users_copy', engine, if_exists='replace')
```

```
# Прямое использование соединения
```

```
with engine.connect() as conn:
```

```
    result = conn.execute("SELECT COUNT(*) FROM users")
```

```
    count = result.scalar()
```

```
Интеграция с Flask
```

```
from flask import Flask, jsonify, request
```

```
import psycopg2
```

```
from psycopg2.extras import RealDictCursor
```

```
app = Flask(__name__)
```

```
# Конфигурация подключения
```

```
DATABASE_CONFIG = {
```

```
    'host': 'localhost',
```

```
    'database': 'myapp',
```

```
    'user': 'postgres',
```

```
    'password': 'password'
```

```
}
```

```
def get_db_connection():
```

```
    return psycopg2.connect(**DATABASE_CONFIG)
```

```
@app.route('/users', methods=['GET'])
```

```
def get_users():
```

```
    with get_db_connection() as conn:
```

```
        with conn.cursor(cursor_factory=RealDictCursor) as cur:
```

```
            cur.execute("SELECT id, name, email FROM users")
```

```
            users = cur.fetchall()
```

```
            return jsonify([dict(user) for user in users])
```

```
@app.route('/users', methods=['POST'])
```

```
def create_user():
```

```
    data = request.get_json()
```

```
    with get_db_connection() as conn:
```

```
        with conn.cursor() as cur:
```

```
            cur.execute(
```

```
                "INSERT INTO users (name, email) VALUES (%s, %s) RETURNING id",
```



```

        (data['name'], data['email'])
    )
    user_id = cur.fetchone()[0]
    return jsonify({'id': user_id, 'message': 'User created'})

if __name__ == '__main__':
    app.run(debug=True)

```

Интеграция с Django

Django использует `psycopg2` по умолчанию для работы с PostgreSQL. Настройка в `settings.py`:

```

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'myproject',
        'USER': 'postgres',
        'PASSWORD': 'password',
        'HOST': 'localhost',
        'PORT': '5432',
        'OPTIONS': {
            'options': '-c default_transaction_isolation=serializable'
        }
    }
}

```

Дополнительные настройки для `psycopg2`

```

DATABASES['default']['OPTIONS']['isolation_level'] =
psycopg2.extensions.ISOLATION_LEVEL_SERIALIZABLE

```

Использование сырых SQL-запросов в Django:

```
from django.db import connection
```

```
def get_user_statistics():
```

```
    with connection.cursor() as cursor:
```

```
        cursor.execute("""
```

```
            SELECT COUNT(*) as total_users,
```

```
                AVG(age) as avg_age,
```

```
                MAX(created_at) as last_registration
```

```
            FROM users
```

```
        """)
```

```
        row = cursor.fetchone()
```

```
        return {
```

```
            'total_users': row[0],
```

```
            'avg_age': row[1],
```

```
            'last_registration': row[2]
```

```
        }
```

Интеграция с FastAPI

```
from fastapi import FastAPI, HTTPException
```

```
from pydantic import BaseModel
```

```
import psycpg2
```

```
from psycpg2.extras import RealDictCursor
```

```
from contextlib import contextmanager
```

```
app = FastAPI()
```

```
DATABASE_URL = "postgresql://user:password@localhost/dbname"
```

@contextmanager

def get_db():

conn = psycopg2.connect(DATABASE_URL)

try:

yield conn

finally:

conn.close()

class UserCreate(BaseModel):

name: str

email: str

age: int

class UserResponse(BaseModel):

id: int

name: str

email: str

age: int

@app.post("/users/", response_model=UserResponse)

async def create_user(user: UserCreate):

with get_db() as conn:

with conn.cursor(cursor_factory=RealDictCursor) as cur:

cur.execute(

"INSERT INTO users (name, email, age) VALUES (%s, %s, %s) RETURNING *",

(user.name, user.email, user.age)

)

result = cur.fetchone()

```
return UserResponse(**result)
```

```
@app.get("/users/{user_id}", response_model=UserResponse)
```

```
async def get_user(user_id: int):
```

```
    with get_db() as conn:
```

```
        with conn.cursor(cursor_factory=RealDictCursor) as cur:
```

```
            cur.execute("SELECT * FROM users WHERE id = %s", (user_id,))
```

```
            result = cur.fetchone()
```

```
            if not result:
```

```
                raise HTTPException(status_code=404, detail="User not found")
```

```
            return UserResponse(**result)
```

Сравнение с другими драйверами PostgreSQL

Критерий	Psycopg2	asyncpg	aiopg	psycopg3
Асинхронность	Нет	Да	Да	Да
Скорость	Высокая	Очень высокая	Средняя	Высокая
Зрелость	Очень высокая	Высокая	Средняя	Средняя
Совместимость с ORM	Отличная	Ограниченная	Ограниченная	Хорошая
Размер сообщества	Очень большое	Большое	Среднее	Растущее
Поддержка типов PostgreSQL	Полная	Полная	Частичная	Полная
Простота использования	Очень простая	Простая	Простая	Простая
Потребление памяти	Умеренное	Низкое	Умеренное	Умеренное
Поддержка Python версий 2.7, 3.6+	3.6+	3.6+	3.6+	3.7+

Когда использовать каждый драйвер:

Psycopg2 - лучший выбор для:

Синхронных приложений

Работы с Django, Flask

Максимальной совместимости

Стабильных production-систем

asynpcrg - лучший выбор для:

Высокопроизводительных асинхронных приложений

Микросервисов с высокой нагрузкой

Когда критична скорость выполнения запросов

psycopg3 - лучший выбор для:

Новых проектов с требованиями асинхронности

Приложений, которым нужна совместимость с ORM

Когда нужны современные возможности Python

Оптимизация производительности

Настройка пула соединений

from psycopg2 import pool

Создание пула соединений

```
connection_pool = psycopg2.pool.ThreadedConnectionPool(  
    minconn=1,  
    maxconn=20,  
    host='localhost',  
    database='mydb',  
    user='postgres',  
    password='password'  
)
```

def execute_query(query, params=None):

conn = connection_pool.getconn()

try:

```
with conn.cursor() as cur:

    cur.execute(query, params)

    return cur.fetchall()

finally:

    connection_pool.putconn(conn)
```

Оптимизация запросов

Использование prepare statements для повторяющихся запросов

```
cur.execute("PREPARE get_user_by_email AS SELECT * FROM users WHERE email = $1")
cur.execute("EXECUTE get_user_by_email (%s)", ("user@example.com",))
```

Использование индексов

```
cur.execute("CREATE INDEX CONCURRENTLY idx_users_email ON users(email)")
```

Анализ планов выполнения

```
cur.execute("EXPLAIN ANALYZE SELECT * FROM users WHERE email = %s",
("user@example.com",))

plan = cur.fetchall()
```

Оптимизация массовых операций

```
from psycopg2.extras import execute_values, execute_batch
```

Эффективная массовая вставка

```
data = [(f"user_{i}", f"user_{i}@example.com") for i in range(1000)]
```

Метод 1: execute_values (самый быстрый)

```
execute_values(
    cur,
    "INSERT INTO users (name, email) VALUES %s",
    data,
    template=None,
```

```
    page_size=100
)
```

Метод 2: execute_batch (средняя скорость)

```
execute_batch(
    cur,
    "INSERT INTO users (name, email) VALUES (%s, %s)",
    data,
    page_size=100
)
```

Метод 3: COPY (для очень больших объемов)

```
from io import StringIO
import csv
```

```
data_file = StringIO()
writer = csv.writer(data_file)
writer.writerows(data)
data_file.seek(0)
```

```
cur.copy_from(data_file, 'users', columns=('name', 'email'), sep=',')
```

Тестирование и отладка

Настройка тестовой среды

```
import pytest
import psycopg2
from psycopg2.extras import DictCursor
```

```
@pytest.fixture
```

```
def db_connection():
```

```
    """Фикстура для подключения к тестовой базе данных"""
```

```
    conn = psycopg2.connect(
```

```
        host='localhost',
```

```
        database='test_db',
```

```
        user='test_user',
```

```
        password='test_password'
```

```
    )
```

```
    yield conn
```

```
    conn.close()
```

```
@pytest.fixture
```

```
def clean_db(db_connection):
```

```
    """Фикстура для очистки базы данных перед каждым тестом"""
```

```
    with db_connection.cursor() as cur:
```

```
        cur.execute("TRUNCATE users RESTART IDENTITY CASCADE")
```

```
    db_connection.commit()
```

```
    yield db_connection
```

Примеры тестов

```
def test_user_creation(clean_db):
```

```
    """Тест создания пользователя"""
```

```
    with clean_db.cursor() as cur:
```

```
        cur.execute(
```

```
            "INSERT INTO users (name, email) VALUES (%s, %s) RETURNING id",
```

```
            ("Test User", "test@example.com")
```

```
        )
```

```
        user_id = cur.fetchone()[0]
```

```
        assert user_id is not None
```



```
assert user_id > 0
```

```
def test_user_retrieval(clean_db):
```

```
    """Тест получения пользователя"""
```

```
    with clean_db.cursor(cursor_factory=DictCursor) as cur:
```

```
        # Создаем пользователя
```

```
        cur.execute(
```

```
            "INSERT INTO users (name, email) VALUES (%s, %s) RETURNING id",
```

```
            ("Test User", "test@example.com")
```

```
        )
```

```
        user_id = cur.fetchone()[0]
```

```
        # Получаем пользователя
```

```
        cur.execute("SELECT * FROM users WHERE id = %s", (user_id,))
```

```
        user = cur.fetchone()
```

```
        assert user['name'] == "Test User"
```

```
        assert user['email'] == "test@example.com"
```

Отладка SQL-запросов

```
import logging
```

```
# Настройка логирования для отладки
```

```
logging.basicConfig(level=logging.DEBUG)
```

```
logger = logging.getLogger(__name__)
```

```
def debug_query(cur, query, params=None):
```

```
    """Функция для отладки SQL-запросов"""
```

```
    logger.debug(f"Выполняется запрос: {query}")
```

if params:

logger.debug(f"Параметры: {params}")

cur.execute(query, params)

logger.debug(f"Затронуто строк: {cur.rowcount}")

logger.debug(f"Статус: {cur.statusmessage}")

return cur.fetchall()

Мониторинг производительности

import time

from contextlib import contextmanager

@contextmanager

def measure_query_time(query_name):

"""Контекстный менеджер для измерения времени выполнения запросов"""

start_time = time.time()

try:

yield

finally:

end_time = time.time()

execution_time = end_time - start_time

print(f"Запрос '{query_name}' выполнен за {execution_time:.4f} секунд")

Использование

with measure_query_time("get_all_users"):

cur.execute("SELECT * FROM users")

users = cur.fetchall()

Работа с большими объемами данных

Серверные курсоры

```
def process_large_dataset(conn, query, batch_size=1000):  
    """Обработка больших наборов данных с помощью серверного курсора"""  
    with conn.cursor(name='large_data_cursor') as cur:  
        cur.execute(query)  
  
        while True:  
            batch = cur.fetchmany(batch_size)  
            if not batch:  
                break  
  
            # Обработка батча  
            for row in batch:  
                process_row(row)  
  
            # Периодическая фиксация для освобождения памяти  
            conn.commit()
```

```
def process_row(row):  
    """Обработка одной строки данных"""  
    # Логика обработки строки  
    pass
```

Использование COPY для больших объемов

```
import csv  
from io import StringIO
```

```
def bulk_import_from_csv(conn, table_name, csv_file_path):  
    """Массовый импорт данных из CSV файла"""
```

```

with open(csv_file_path, 'r', encoding='utf-8') as file:

    with conn.cursor() as cur:

        cur.copy_expert(

            f"COPY {table_name} FROM STDIN WITH CSV HEADER",

            file

        )

    conn.commit()

```

```

def bulk_export_to_csv(conn, table_name, csv_file_path):

    """Массовый экспорт данных в CSV файл"""

    with open(csv_file_path, 'w', encoding='utf-8', newline='') as file:

        with conn.cursor() as cur:

            cur.copy_expert(

                f"COPY {table_name} TO STDOUT WITH CSV HEADER",

                file

            )

```

Потоковая обработка данных

```

def stream_process_data(conn, query, processor_func):

    """Потоковая обработка данных без загрузки в память"""

    with conn.cursor(name='stream_cursor') as cur:

        cur.execute(query)

        for row in cur:

            try:

                processor_func(row)

            except Exception as e:

                logger.error(f"Ошибка обработки строки {row}: {e}")

                continue

```

Часто задаваемые вопросы

Что такое psycopg2 и зачем он нужен?

Psycopg2 — это популярный адаптер PostgreSQL для Python, который обеспечивает надежное и эффективное взаимодействие между Python-приложениями и базой данных PostgreSQL. Он необходим для выполнения SQL-запросов, управления транзакциями и работы с различными типами данных PostgreSQL.

Как правильно обрабатывать ошибки подключения?

```
import psycopg2

from psycopg2 import OperationalError, InterfaceError

try:
    conn = psycopg2.connect(
        host='localhost',
        database='mydb',
        user='postgres',
        password='password',
        connect_timeout=10
    )
except OperationalError as e:
    print(f"Ошибка подключения: {e}")
except InterfaceError as e:
    print(f"Ошибка интерфейса: {e}")
except Exception as e:
    print(f"Неожиданная ошибка: {e}")
```

Поддерживает ли psycopg2 транзакции?

Да, **psycopg2** полностью поддерживает транзакции. По умолчанию каждая операция выполняется в рамках транзакции, которую необходимо подтвердить с помощью **commit()** или отменить с помощью **rollback()**.

Как защитить приложение от SQL-инъекций?

Всегда используйте параметризованные запросы с плейсхолдерами %s вместо форматирования строк:

ПРАВИЛЬНО

```
cur.execute("SELECT * FROM users WHERE email = %s", (email,))
```

НЕПРАВИЛЬНО

```
cur.execute(f"SELECT * FROM users WHERE email = '{email}'")
```

Можно ли использовать psycopg2 с Django?

Да, psycopg2 является рекомендуемым драйвером для работы с PostgreSQL в Django. Он используется автоматически при настройке ENGINE: 'django.db.backends.postgresql' в настройках базы данных.

Есть ли у psycopg2 поддержка асинхронности?

Нет, psycopg2 является синхронным драйвером. Для асинхронных приложений рекомендуется использовать asyncpg или новую версию psycopg3, которая поддерживает как синхронный, так и асинхронный API.

Как работать с psycopg2 в многопоточных приложениях?

Psycopg2 поддерживает многопоточность на уровне соединений. Рекомендуется использовать пул соединений и не разделять одно соединение между потоками:

```
from psycopg2 import pool
```

```
connection_pool = pool.ThreadedConnectionPool(  
    minconn=1,  
    maxconn=20,  
    host='localhost',  
    database='mydb',
```

```
user='postgres',  
password='password'  
)
```

Как оптимизировать производительность при работе с большими объемами данных?

Используйте следующие методы:

Серверные курсоры для обработки больших наборов данных

execute_values() для массовых вставок

COPY для импорта/экспорта больших файлов

Пул соединений для уменьшения накладных расходов на подключение

Поддерживает ли psycopg2 работу с JSON данными?

Да, psycopg2 автоматически преобразует JSON и JSONB данные PostgreSQL в Python словари и списки. Также можно использовать psycopg2.extras.Json для явного указания JSON данных.

Как правильно закрывать соединения?

Всегда закрывайте соединения и курсоры после использования. Рекомендуется использовать контекстные менеджеры:

```
with psycopg2.connect(dsn) as conn:  
    with conn.cursor() as cur:  
        cur.execute("SELECT * FROM users")  
        # Соединение и курсор закроются автоматически
```

Заключение

Psycopg2 остается золотым стандартом для работы с PostgreSQL в Python благодаря своей надежности, производительности и богатой функциональности. Эта библиотека предоставляет все необходимые инструменты для создания надежных и масштабируемых приложений, работающих с данными.

Основные преимущества psycopg2:

Стабильность и зрелость кода

Полная совместимость с PostgreSQL

Отличная производительность

Богатый функционал для работы с различными типами данных

Широкая поддержка в экосистеме Python